

CH12 — Common Pointer Problems

SLIDE 1 — What Can Go Wrong With Pointers?

Pointers are useful, but small mistakes can cause problems.

Common pointer mistakes include:

- Using an **uninitialized pointer**
- Dereferencing a **NULL pointer**
- Using the **wrong pointer type**
- Accessing an **invalid memory location**

Example:

```
C/C++
int *p;

printf("%d", *p);
```

Here, **p** does not point to a valid variable.

Easy Analogy:

A pointer is like an address written on paper.

If the address is **missing or incorrect**, you cannot safely find the house.

Remember:

VALID ADDRESS → SAFE POINTER

SLIDE 2 — Uninitialized Pointer

An uninitialized pointer does not have a valid address assigned to it.

Example:

C/C++

```
int *p;
```

```
*p = 10;
```

✗ This is dangerous because `p` does not point to a known variable.

✓ **Correct way:**

C/C++

```
int x;
```

```
int *p = &x;
```

```
*p = 10;
```

Now:

None

```
x = 10
```

↑

```
p
```

💡 **Memory Trick:**

CREATE → ADDRESS → USE

First give the pointer a valid address before using `*p`.

SLIDE 3 — NULL Pointer

A NULL pointer does not point to a valid object.

Example:

C/C++

```
int *p = NULL;
```

We should **not** dereference it:

C/C++

```
*p = 10;
```

✗ This can cause a program crash or undefined behavior.

Check before using:

C/C++

```
if (p != NULL)
{
    printf("%d", *p);
}
```

Easy Analogy:

NULL is like saying:

"I don't have an address."

You cannot go to a house when there is no address.

★ Remember:

NULL → NO VALID ADDRESS

SLIDE 4 — Dangling Pointer

A **dangling pointer** is a pointer that refers to memory that is no longer valid to use.

For example, a pointer may point to a variable whose lifetime has ended.

C/C++

```
int *p;
```

```
{  
    int x = 10;  
    p = &x;  
}
```

After the block ends, `x` no longer exists.

So using:

```
C/C++  
*p
```

is unsafe.

Easy Analogy

Imagine having the address of a house that has already been demolished.

The address exists on paper, but the house is no longer there.

 **Memory Trick:**

VALID OBJECT → USE POINTER
OBJECT GONE → DON'T USE POINTER

SLIDE 5 — RECAP

Common Pointer Problems

Problem	Meaning
Uninitialized pointer	Pointer has not been given a valid address
NULL pointer	Pointer intentionally points to no valid object
Dangling pointer	Pointer refers to an object whose lifetime has ended

Invalid access

Trying to access memory through an invalid pointer

BIG MEMORY TRICK

CHECK → POINT → USE

None

Is the pointer valid?

↓

Does it point to a valid object?

↓

Now use `*p`

★ **TAKEAWAY:**

Before dereferencing a pointer, make sure it points to a **valid object**.

VALID POINTER → SAFE ACCESS